Summary

(a) Hardware interaction can happen in two ways: (1) When the user interacts with the hardware and the program reacts to it. (2) When the program interacts with the hardware without any user intervention.
(b) In DOS when the user interacts with the hardware an ISR gets called which interacts with the hardware. In Windows the same thing is done by the device driver's ISR.
(c) In DOS when the program has to interact with the hardware it can do so by using library functions, DOS/BIOS routines or by directly interacting with the hardware. In Windows the same thing can be done by using API functions.
(d) Under Windows to gain finer control over the hardware we are required to write a device driver program.
(e) Interaction with the any device can be done using API functions like **CreateFile( )**, **ReadFile( )**, **WriteFile( )** and **CloseHandle( )**.
(f) Different strings have to be passed to the **CreateFile( )** functions for interacting with different devices.
(g) Windows provides a powerful mechanism called hooks that can alter the flow of messages before they reach the application.
(h) Windows hook procedures should be written in a DLL since they work on a system wide basis.
(i) Windows hooks can be put to many good uses.

Exercise

[A] State True or False:

(a) In MS-DOS on occurrence of an interrupt values from IDT are used to call the appropriate kernel routine.
(b) Under Windows on occurrence of an interrupt the kernel routine calls the appropriate device driver's ISR.
(c) Under Windows an application can interact with the hardware by directly calling its device driver's routines.

(d) Under Windows we can write device drivers to extend the OS itself.
(e) **ReadSector( )** and **WriteSector( )** are API functions.
(f) While reading a sector from the disk the **CreateFile( )** function creates a file on the disk.
(g) The Windows API function to stop communication with a device is **CloseFile( )**.
(h) The **ReadFile( )** and **WriteFile( )** API functions can only perform reading or writing from/to a disk file.

[B] Answer the following:

(a) How is hardware interaction under Windows different that that under DOS?

(b) What is the advantage of writing code in a DLL?

(c) Explain the Windows hooks mechanism.

(d) What is the standard way of communicating with a device under Windows?

(e) Write a program to read the contents of Boot Sector of a 32-bit FAT file system and print them on the screen. Refer Appendix G for details about the contents of the boot sector.

(f) Write a program that ensures that the key 'A' is completely disabled across all applications.

(g) Write a program that closes any window just by placing the cursor on the 'Close' button in the title bar of it.

20 C Under Linux

- What is Linux
- C Programming Under Linux
- The ‘Hello Linux’ Program
- Processes
- Parent and Child Processes
- More Processes
- Zombies and Orphans
- One Interesting Fact
- Summary
- Exercise

Today the programming world is divided into two major camps—the Windows world and the Linux world. Since its humble beginning about a decade ago, Linux has steadily drawn the attention of programmers across the globe and has successfully created a community of its own. How big and committed is this community is one of the hottest debates that is raging in all parts of the world. You can look at the hot discussions and the flame wars on this issue on numerous sites on the internet. Before you decide to join the Windows or the Linux camp you should first get familiar with both of them. The last 4 chapters concentrated on Windows programming. This and the next one would deal with Linux programming. Without any further discussions let us now set out on the Linux voyage. I hope you find the journey interesting and exciting.

What is Linux

Linux is a clone of the Unix operating system. Its kernel was written from scratch by Linus Torvalds with assistance from a loosely-knit team of programmers across the world on Internet. It has all the features you would expect in a modern OS. Moreover, unlike Windows or Unix, Linux is available completely free of cost. The kernel of Linux is available in source code form. Anybody is free to change it to suit his requirement, with a precondition that the changed kernel can be distributed only in the source code form. Several programs, frameworks, utilities have been built around the Linux kernel. A common user may not want the headaches of downloading the kernel, going through the complicated compilation process, then downloading the frameworks, programs and utilities. Hence many organizations have come forward to make this job easy. They distribute the precompiled kernel, programs, utilities and frameworks on a common media. Moreover, they also provide installation scripts for easy installations of the Linux OS and applications. Some of the popular distributions are RedHat, SUSE, Caldera, Debian, Mandrake, Slackware, etc. Each of them contain the same kernel

but may contain different application programs, libraries, frameworks, installation scripts, utilities, etc. Which one is better than the other is only a matter of taste.

Linux was first developed for x86-based PCs (386 or higher). These days it also runs on Compaq Alpha AXP, Sun SPARC, Motorola 68000 machines (like Atari ST and Amiga), MIPS, PowerPC, ARM, Intel Itanium, SuperH, etc. Thus Linux works on literally every conceivable microprocessor architecture.

Under Linux one is faced with simply too many choices of Linux distributions, graphical shells and managers, editors, compilers, linkers, debuggers, etc. For simplicity (in my opinion) I have chosen the following combination:

Linux Distribution - Red Hat Linux 9.0
Console Shell - BASH
Graphical Shell - KDE 3.1-10
Editor - KWrite
Compiler - GNU C and C++ compiler (gcc)

We would be using and discussing these in the sections to follow.

C Programming Under Linux

How is C under Linux any different than C under DOS or C under Windows? Well, it is same as well as different. It is same to the extent of using language elements like data types, control instructions and the overall syntax. The usage of standard library functions is also same even though the implementation of each might be different under different OS. For example, a **printf( )** would work under all OSs, but the way it is defined is likely to be different for different OSs. The programmer however doesn't suffer because of this since he can continue to call **printf( )** the same way no matter how it is implemented.

But there the similarity ends. If we are to build programs that utilize the features offered by the OS then things are bound to be different across OSs. For example, if we are to write a C program that would create a Window and display a message "hello" at the point where the user clicks the left mouse button. The architecture of this program would be very closely tied with the OS under which it is being built. This is because the mechanisms for creating a window, reporting a mouse click, handling a mouse click, displaying the message, closing the window, etc. are very closely tied with the OS for which the program is being built. In short the programming architecture (better known as programming model) for each OS is different. Hence naturally the program that achieves the same task under different OS would have to be different.

The 'Hello Linux' Program

As with any new platform we would begin our journey in the Linux world by creating a 'hello world' program. Here is the source code....

```
int main( )
{
    printf ( "Hello Linux\n" ) ;
    return 0 ;
}
```

The program is exactly same as compared to a console program under DOS/Windows. It begins with **main( )** and uses **printf( )** standard library function to produce its output. So what is the difference? The difference is in the way programs are typed, compiled and executed. The steps for typing, compiling and executing the program are discussed below.

The first hurdle to cross is the typing of this program. Though any editor can be used to do so, we have preferred to use the editor called 'KWrite'. This is because it is a very simple yet elegant

editor compared to other editors like 'vi' or 'emacs'. Note that KWrite is a text editor and is a part of K Desktop environment (KDE). Installation of Linux and KDE is discussed in Appendix H. Once KDE is started select the following command from the desktop panel to start KWrite:

K Menu | Accessories | More Accessories | KWrite

If you face any difficulty in starting the KWrite editor please refer Appendix H. Assuming that you have been able to start KWrite successfully, carry out the following steps:

(a) Type the program and save it under the name 'hello.c'.
(b) At the command prompt switch to the directory containing 'hello.c' using the **cd** command.
(c) Now compile the program using the **gcc** compiler as shown below:

```
# gcc hello.c
```

(d) On successful compilation **gcc** produces a file named 'a.out'. This file contains the machine code of the program which can now be executed.
(e) Execute the program using the following command.

```
# ./a.out
```

(f) Now you should be able to see the output 'Hello Linux' on the screen.

Having created a Hello Linux program and gone through the edit-compile-execute cycle once let us now turn our attention to Linux specific programming. We will begin with processes.

Processes

Gone are the days when only one job (task) could be executed in memory at any time. Today the modern OSs like Windows and

Linux permit execution of several tasks simultaneously. Hence these OSs are aptly called 'Multitasking' OSs.

In Linux each running task is known as a 'process'. Even though it may appear that several processes are being executed by the microprocessor simultaneously, in actuality it is not so. What happens is that the microprocessor divides the execution time equally among all the running processes. Thus each process gets the microprocessor's attention in a round robin manner. Once the time-slice allocated for a process expires the operation that it is currently executing is put on hold and the microprocessor now directs its attention to the next process. Thus at any given moment if we take the snapshot of memory only one process is being executed by the microprocessor. The switching of processes happens so fast that we get a false impression that the processor is executing several processes simultaneously.

The scheduling of processes is done by a program called 'Scheduler' which is a vital component of the Linux OS. This scheduler program is fairly complex. Before switching over to the next thread it stores the information about the current process. This includes current values of CPU registers, contents of System Stack and Application Stack, etc. When this process again gets the time slot these values are restored. This process of shifting over from one thread to another is often called a Context Switch. Note that Linux uses preemptive scheduling, meaning thereby that the context switch is performed as soon as the time slot allocated to the process is over, no matter whether the process has completed its job or not.

Kernel assigns each process running in memory a unique ID to distinguish it from other running processes. This ID is often known as processes ID or simply PID. It is very simple to print the PID of a running process programmatically. Here is the program that achieves this…

```
int main( )
{
    printf ( "Process ID = %d", getpid( ) ) ;
}
```

Here **getpid( )** is a library function which returns the process ID of the calling process. When the execution of the program comes to an end the process stands terminated. Every time we run the program a new process is created. Hence the kernel assigns a new ID to the process each time. This can be verified by executing the program several times—each time it would produce a different output.

Parent and Child Processes

As we know, our running program is a process. From this process we can create another process. There is a parent-child relationship between the two processes. The way to achieve this is by using a library function called **fork( )**. This function splits the running process into two processes, the existing one is known as parent and the new process is known as child. Here is a program that demonstrates this…

```
# include <sys/types.h>
int main( )
{
    printf ( "Before Forking\n" ) ;
    fork( ) ;
    printf ( "After Forking\n" ) ;
}
```

Here is the output of the program…

```
Before Forking
After Forking
After Forking
```

Watch the output of the program. You can notice that all the statements after the **fork( )** are executed twice—once by the parent process and second time by the child process. In other words **fork( )** has managed to split our process into two.

But why on earth would we like to do this? At times we want our program to perform two jobs simultaneously. Since these jobs may be inter-related we may not want to create two different programs to perform them. Let me give you an example. Suppose we want perform two jobs—copy contents of source file to target file and display an animated GIF file indicating that the file copy is in progress. The GIF file should continue to play till file copy is taking place. Once the copying is over the playing of the GIF file should be stopped. Since both these jobs are inter-related they cannot be performed in two different programs. Also, they cannot be performed one after another. Both jobs should be performed simultaneously.

At such times we would want to use **fork( )** to create a child process and then write the program in such a manner that file copy is done by the parent and displaying of animated GIF file is done by the child process. The following program shows how this can be achieved. Note that the issue here is to show how to perform two different but inter-related jobs simultaneously. Hence I have skipped the actual code for file copying and playing the animated GIF file.

```
# include <sys/types.h>

int main( )
{
    int  pid ;
    pid = fork( ) ;
    if ( pid == 0 )
    {
        printf ( "In child process\n" ) ;
        /* code to play animated GIF file */
```

```
    }
    else
    {
        printf ( "In parent process\n" ) ;
        /* code to copy file */
    }
}
```

As we know, **fork( )** creates a child process and duplicates the code of the parent process in the child process. There onwards the execution of the **fork( )** function continues in both the processes. Thus the duplication code inside **fork( )** is executed once, whereas the remaining code inside it is executed in both the parent as well as the child process. Hence control would come back from **fork( )** twice, even though it is actually called only once. When control returns from **fork( )** of the parent process it returns the PID of the child process, whereas when control returns from **fork( )** of the child process it always returns a 0. This can be exploited by our program to segregate the code that we want to execute in the parent process from the code that we want to execute in the child process. We have done this in our program using an **if** statement. In the parent process the 'else block' would get executed, whereas in the child process the 'if block' would get executed.

Let us now write one more program. This program would use the **fork( )** call to create a child process. In the child process we would print the PID of child and its parent, whereas in the parent process we would print the PID of the parent and its child. Here is the program…

```
# include <sys/types.h>
int main( )
{
    int pid ;
    pid = fork( ) ;

    if ( pid == 0 )
```

```
    {
        printf ( "Child : Hello I am the child process\n" ) ;
        printf ( "Child : Child's PID: %d\n", getpid( ) ) ;
        printf ( "Child : Parent's PID: %d\n", getppid( ) ) ;
    }
    else
    {
        printf ( "Parent : Hello I am the parent process\n" ) ;
        printf ( "Parent : Parent's PID: %d\n", getpid( ) ) ;
        printf ( "Parent : Child's PID: %d\n", pid ) ;
    }
}
```

Given below is the output of the program:

```
Child : Hello I am the child process
Child : Child's PID: 4706
Child : Parent's PID: 4705
Parent : Hello I am the Parent process
Parent : Parent's PID: 4705
Parent : Child's PID: 4706
```

In addition to **getpid( )** there is another related function that we have used in this program—**getppid( )**. As the name suggests, this function returns the PID of the parent of the calling process.

You can tally the PIDs from the output and convince yourself that you have understood the **fork( )** function well. A lot of things that follow use the **fork( )** function. So make sure that you understand it thoroughly.

Note that even Linux internally uses **fork( )** to create new child processes. Thus there is a inverted tree like structure of all the processes running in memory. The father of all these processes is a process called **init**. If we want to get a list of all the running processes in memory we can do so using the **ps** command as shown below.

```
# ps –A
```

Here the switch **–A** indicates that we want to list all the running processes.

More Processes

Suppose we want to execute a program on the disk as part of a child process. For this first we should create a child process using **fork( )** and then from within the child process we should call an **exec** function to execute the program on the disk as part of a child process. Note that there is a family of **exec** library functions, each basically does the same job but with a minor variation. For example, **execl( )** function permits us to pass a list of command line arguments to the program to be executed. **execv( )** also does the same job as **execl( )** except that the command line arguments can be passed to it in the form of an array of pointers to strings. There also exist other variations like **execle( )** and **execvp( )**.

Let us now see a program that uses **execl( )** to run a new program in the child process.

```
# include <unistd.h>
int main( )
{
    int pid ;
    pid = fork( ) ;
    if ( pid == 0 )
    {
        execl ( "/bin/ls","-al", "/etc", NULL ) ;
        printf ( "Child: After exec( )\n") ;
    }
    else
        printf ( "Parent process\n" ) ;
}
```

After forking a child process we have called the **execl( )** function. This function accepts variable number of arguments. The first parameter to **execl( )** is the absolute path of the program to be executed. The remaining parameters describe the command line arguments for the program to be executed. The last parameter is an end of argument marker which must always be **NULL**. Thus in our case the we have called upon the **execl( )** function to execute the **ls** program as shown below

```
ls -al /etc
```

As a result, all the contents of the **/etc** directory are listed on the screen. Note that the **printf( )** below the call to **execl( )** function is not executed. This is because the **exec** family functions overwrite the image of the calling process with the code and data of the program that is to be executed. In our case the child process's memory was overwritten by the code and data of the **ls** program. Hence the call to **printf( )** did not materialize.

It would make little sense in calling **execl( )** before **fork( )**. This is because a child would not get created and **execl( )** would simply overwrite the main process itself. As a result, no statement beyond the call to **execl( )** would ever get executed. Hence **fork( )** and **execl( )** usually go hand in hand.

Zombies and Orphans

We know that the **ps –A** command lists all the running processes. But from where does the **ps** program get this information? Well, Linux maintains a table containing information about all the processes. This table is called 'Process Table'. Apart from other information the process table contains an entry of 'exit code' of the process. This integer value indicates the reason why the process was terminated. Even though the process comes to an end its entry would remain in the process table until such time that the parent of the terminated process queries the exit code. This act of querying

deletes the entry of the terminated process from the process table and returns the exit code to the parent that raised the query.

When we fork a new child process and the parent and the child continue to execute there are two possibilities—either the child process ends first or the parent process ends first. Let us discuss both these possibilities.

(a) Child terminates earlier than the parent

In this case till the time parent does not query the exit code of the terminated child the entry of the child process would continue to exist. Such a process in Linux terminology is known as a 'Zombie' process. Zombie means ghost, or in plain simple Hindi a 'Bhoot'. Moral is, a parent process should query the process table immediately after the child process has terminated. This would prevent a zombie.

What if the parent terminates without querying. In such a case the zombie child process is treated as an 'Orphan' process. Immediately, the father of all processes—**init**—adopts the orphaned process. Next, as a responsible parent **init** queries the process table as a result of which the child process entry is eliminated from the process table.

(b) Parent terminates earlier than the child

Since every parent process is launched from the Linux shell, the parent of the parent is the **shell** process. When our parent process terminates, the **shell** queries the process table. Thus a proper cleanup happens for the parent process. However, the child process which is still running is left orphaned. Immediately the **init** process would adopt it and when its execution is over **init** would query the process table to clean up the entry for the child process. Note that in this case the child process does not become a zombie.

Thus, when a zombie or an orphan gets created the OS takes over and ensures that a proper cleanup of the relevant process table

entry happens. However, as a good programming practice our program should get the exit code of the terminated process and thereby ensure a proper cleanup. Note that here cleanup is important (it happens anyway). Why is it important to get the exit code of the terminated process. It is because, it is the exit code that would give indication about whether the job assigned to the process was completed successfully or not. The following program shows how this can be done.

```
# include <unistd.h>
# include <sys/types.h>
int main( )
{
    unsigned int i = 0 ;
    int pid, status ;
    pid = fork( ) ;
    if ( pid == 0 )
    {
        while ( i < 4294967295U )
            i++ ;
        printf ( "The child is now terminating\n" ) ;
    }
    else
    {
        waitpid ( pid, &status, 0 )  ;
        if ( WIFEXITED ( status ) )
            printf ( "Parent: Child terminated normally\n" ) ;
        else
            printf ( "Parent: Child terminated abnormally\n" ) ;
    }
    return 0 ;
}
```

In this program we have applied a big loop in the child process. This loop ensures that the child does not terminate immediately. From within the parent process we have made a call to the **waitpid( )** function. This function makes the parent process wait

till the time the execution of the child process does not come to an end. This ensures that the child process never becomes orphaned. Once the child process, terminates the **waitpid( )** function queries its exit code and returns back to the parent. As a result of querying, the child process does not become a zombie.

The first parameter of **waitpid( )** function is the pid of the child process for which the wait has to be performed. The second parameter is the address of an integer variable which is set up with the exit status code of the child process. The third parameter is used to specify some options to control the behavior of the wait operation. We have not used this parameter and hence we have passed a **0**. Next we have made use of the **WIFEXITED( )** macro to test if the child process exited normally or not. This macro takes the status value as a parameter and returns a non-zero value if the process terminated normally. Using this macro the parent suitably prints a message to report the status (normal/abnormal) termination of its child process.

One Interesting Fact

When we use **fork( )** to create a child process the child process does not contain the entire data and code of the parent process. Then does it mean that the child process contains the data and code below the **fork( )** call. Even this is not so. In actuality the code never gets duplicated. Linux internally manages to intelligently share it. As against this, some data is shared, some is not. Till the time both the processes do not change the value of the variables they keep getting shared. However, if any of the processes (either child or parent) attempt to change the value of a variable it is no longer shared. Instead a new copy of the variable is made for the process that is attempting to change it. This not only ensures data integrity but also saves precious memory.

Summary

(a) Linux is a free OS whose kernel was built by Linus Trovalds and friends.
(b) A Linux distribution consists of the kernel with source code along with a large collection of applications, libraries, scripts, etc.
(c) C programs under Linux can be compiled using the popular **gcc** compiler.
(d) Basic scheduling unit in Linux is a 'Process'. Processes are scheduled by a special program called 'Scheduler'.
(e) **fork( )** library function can be used to create child processes.
(f) **Init** process is the father of all processes.
(g) **execl( )** library function is used to execute another program from within a running program,.
(h) **execl( )** function overwrites the image (code and data) of the calling process.
(i) **execl( )** and **fork( )** usually go hand in hand.
(j) **ps** command can be used to get a list of all processes.
(k) **kill** command can be used to terminate a process.
(l) A 'Zombie' is a child process that has terminated but its parent is running and has not called a function to get the exit code of the child process.
(m) An 'Orphan' is a child process whose parent has terminated.
(n) Orphaned processes are adopted by **init** process automatically.
(o) A parent process can avoid creation of a Zombie and Orphan processes using **waitpid( )** function.

Exercise

[A] State True or False:

(a) We can modify the kernel of Linux OS.
(b) All distributions of Linux contain the same collection of applications, libraries and installation scripts.
(c) Basic scheduling unit in Linux is a file.

(d) **execl( )** library function can be used to create a new child process.
(e) The scheduler process is the father of all processes.
(f) A family of **fork( )** and **exec( )** functions are available, each doing basically the same job but with minor variations.
(g) **fork( )** completely duplicates the code and data of the parent process into the child process.
(h) **fork( )** overwrites the image (code and data) of the calling process.
(i) **fork( )** is called twice but returns once.
(j) Every zombie process is essentially an orphan process.
(k) Every orphan process is essentially an orphan process.

[B] Answer the following:

(a) If a program contains four calls to **fork( )** one after the other how many total processes would get created?

(b) What is the difference between a zombie process and an orphan process?

(c) Write a program that prints the command line arguments that it receives. What would be the output of the program if the command line argument is * ?

(d) What purpose do the functions **getpid( )**, **getppid( )**, **getpppid( )** serve?

(e) Rewrite the program in the section 'Zombies and Orphans' replacing the **while** loop with a call to the **sleep( )** function. Do you observe any change in the output of the program?

(f) How does **waitpid( )** prevent creation of Zombie or Orphan processes?

21 More Linux Programming

- Communication using Signals
- Handling Multiple Signals
- Registering a Common Handler
- Blocking Signals
- Event driven programming
- Where Do You Go From Here
- Summary
- Exercise

Communication is the essence of all progress. This is true in real life as well as in programming. In today's world a program that runs in isolation is of little use. A worthwhile program has to communicate with the outside world in general and with the OS in particular. In Chapters 16 and 17 we saw how a Windows based program communicates with Windows. In this chapter let us explore how this communication happens under Linux.

Communication using Signals

In the last chapter we used **fork( )** and **exec( )** library function to create a child process and to execute a new program respectively. These library functions got the job done by communication with the Linux OS. Thus the direction of communication was from the program to the OS. The reverse communication—from the OS to the program—is achieved using a mechanism called 'Signal'. Let us now write a simple program that would help you experience the signal mechanism.

```
int main( )
{
    while ( 1 )
        printf ( "Pogram Running\n" ) ;
    return 0 ;
}
```

The program is fairly straightforward. All that we have done here is we have used an infinite **while** loop to print the message "Program Running" on the screen. When the program is running we can terminate it by pressing the Ctrl + C. When we press Ctrl + C the keyboard device driver informs the Linux kernel about pressing of this special key combination. The kernel reacts to this by sending a signal to our program. Since we have done nothing to handle this signal the default signal handler gets called. In this

default signal handler there is code to terminate the program. Hence on pressing Ctrl + C the program gets terminated.

But how on earth would the default signal handler get called. Well, it is simple. There are several signals that can be sent to a program. A unique number is associated with each signal. To avoid remembering these numbers, they have been defined as macros like **SIGINT**, **SIGKILL**, **SIGCONT**, etc. in the file 'signal.h'. Every process contains several 'signal ID - function pointer' pairs indicating for which signal which function should be called. If we do not decide to handle a signal then against that signal ID the address of the default signal handler function is present. It is precisely this default signal handler for **SIGINT** that got called when we pressed Ctrl + C when the above program was executed. INT in **SIGINT** stands for interrupt.

Let us know see how can we prevent the termination of our program even after hitting Ctrl + C. This is shown in the following program:

```
# include <signal.h>

void sighandler ( int signum )
{
    printf ( "SIGINT received. Inside sighandler\n" ) ;
}

int main( )
{
    signal ( SIGINT, ( void* ) sighandler ) ;
    while ( 1 )
        printf ( "Program Running\n" ) ;
    return 0 ;
}
```

In this program we have registered a signal handler for the SIGINT signal by using the **signal( )** library function. The first parameter

of this function specifies the ID of the signal that we wish to register. The second parameter is the address of a function that should get called whenever the signal is received by our program. This address has to be typecasted to a **void *** before passing it to the **signal( )** function.

Now when we press Ctrl + C the registered handler, namely, **sighandler( )** would get called. This function would display the message 'SIGINT received. Inside sighandler' and return the control back to **main( )**. Note that unlike the default handler, our handler does not terminate the execution of our program. So only way to terminate it is to kill the running process from a different terminal. For this we need to open a new instance of command prompt (terminal). How to start a new instace of command prompt is discussed in Appendix H. Next do a **ps –a** to obtain the list of processes running at all the command prompts that we have launched. Note down the process id of **a.out**. Finally kill 'a.out' process by saying

```
# kill 3276
```

In my case the terminal on which I executed **a.out** was **tty1** and its process id turned out to be **3276**. In your case the terminal name and the process id might be a different number.

If we wish we can abort the execution of the program in the signal handler itself by using the **exit (0)** beyond the **printf( )**.

Note that signals work asynchronously. That is, when a signal is received no matter what our program is doing, the signal handler would immediately get called. Once the execution of the signal handler is over the execution of the program is resumed from the point where it left off when the signal was received.

Handling Multiple Signals

Now that we know how to handle one signal, let us try to handle multiple signals. Here is the program to do this…

```
# include <unistd.h>
# include <sys/types.h>
# include <signal.h>

void inthandler ( int signum )
{
     printf ( "\nSIGINT Received\n" ) ;
}

void termhandler ( int signum )
{
     printf ( "\nSIGTERM Received\n" ) ;
}

void conthandler ( int signum )
{
     printf ( "\nSIGCONT Received\n" ) ;
}

int main( )
{
     signal ( SIGINT, inthandler ) ;
     signal ( SIGTERM, termhandler ) ;
     signal ( SIGCONT, conthandler ) ;

     while ( 1 )
          printf ( "\rProgram Running" ) ;

     return 0 ;
}
```

In this program apart from **SIGINT** we have additionally registered two new signals, namely, **SIGTERM** and **SIGCONT**. The **signal( )** function is called thrice to register a different handler for each of the three signals. After registering the signals we enter a infinite **while** loop to print the 'Program running' message on the screen.

As in the previous program, here too, when we press Ctrl + C the handler for the SIGINT i.e. **inthandler( )** is called. However, when we try to kill the program from the second terminal using the **kill** command the program does not terminate. This is because when the **kill** command is used it sends the running program a **SIGTERM** signal. The default handler for the message terminates the program. Since we have handled this signal ourselves, the handler for **SIGTERM** i.e. **termhandler( )** gets called. As a result the **printf( )** statement in the **termhandler( )** function gets executed and the message 'SIGTERM Received' gets displayed on the screen. Once the execution of **termhandler( )** function is over the program resumes its execution and continues to print 'Program Running'. Then how are we supposed to terminate the program? Simple. Use the following command from the another terminal:

```
kill –SIGKILL 3276
```

As the command indicates, we are trying to send a **SIGKILL** signal to our program. A **SIGKILL** signal terminates the program.

Most signals may be caught by the process, but there are a few signals that the process cannot catch, and they cause the process to terminate. Such signals are often known as un-catchable signals. The **SIGKILL** signal is an un-catchable signal that forcibly terminates the execution of a process.

Note that even if a process attempts to handle the **SIGKILL** signal by registering a handler for it still the control would always land in the default **SIGKILL** handler which would terminate the program.

The **SIGKILL** signal is to be used as a last resort to terminate a program that gets out of control. One such process that makes uses of this signal is a system shutdown process. It first sends a **SIGTERM** signal to all processes, waits for a while, thus giving a 'grace period' to all the running processes. However, after the grace period is over it forcibly terminates all the remaining processes using the **SIGKILL** signal.

That leaves only one question—when does a process receive the **SIGCONT** signal? Let me try to answer this question.

A process under Linux can be suspended using the Ctrl + Z command. The process is stopped but is not terminated, i.e. it is suspended. This gives rise to the un-catchable **SIGSTOP** signal. To resume the execution of the suspended process we can make use of the **fg** (foreground) command. As a result of which the suspended program resumes its execution and receives the **SIGCONT** signal (CONT means continue execution).

Registering a Common Handler

Instead of registering a separate handler for each signal we may decide to handle all signals using a common signal handler. This is shown in the following program:

```
# include <unistd.h>
# include <sys/types.h>
# include <signal.h>

void sighandler ( int signum )
{
    switch ( signum )
    {
        case SIGINT :
```

```
            printf ( "SIGINT Received\n" ) ;
            break ;

    case SIGTERM :
            printf ( "SIGTERM Received\n" ) ;
            break ;

    case SIGCONT :
            printf ( "SIGCONT Received\n" ) ;
            break ;
    }
}

int main( )
{
    signal ( SIGINT,  sighandler ) ;
    signal ( SIGTERM, sighandler ) ;
    signal ( SIGCONT, sighandler ) ;

    while ( 1 )
        printf ( "\rProgram running" ) ;

    return 0 ;
}
```

In this program during each call to the **signal( )** function we have specified the address of a common signal handler named **sighandler( )**. Thus the same signal handler function would get called when one of the three signals are received. This does not lead to a problem since the **sighandler( )** we can figure out inside the signal ID using the first parameter of the function. In our program we have made use of the **switch-case** construct to print a different message for each of the three signals.

Note that we can easily afford to mix the two methods of registering signals in a program. That is, we can register separate signal handlers for some of the signals and a common handler for

some other signals. Registering a common handler makes sense if we want to react to different signals in exactly the same way.

Blocking Signals

Sometimes we may want that flow of execution of a critical/time-critical portion of the program should not be hampered by the occurrence of one or more signals. In such a case we may decide to block the signal. Once we are through with the critical/time-critical code we can unblock the signals(s). Note that if a signal arrives when it is blocked it is simply queued into a signal queue. When the signals are unblocked the process immediately receives all the pending signals one after another. Thus blocking of signals defers the delivery of signals to a process till the execution of some critical/time-critical code is over. Instead of completely ignoring the signals or letting the signals interrupt the execution, it is preferable to block the signals for the moment and deliver them some time later. Let us now write a program to understand signal blocking. Here is the program…

```
# include <unistd.h>
# include <sys/types.h>
# include <signal.h>
# include <stdio.h>

void sighandler ( int signum )
{
    switch ( signum )
    {
        case SIGTERM :
            printf ( "SIGTERM Received\n" ) ;
            break ;

        case SIGINT :
            printf ( "SIGINT Received\n" ) ;
            break ;
```

```
            case SIGCONT :
                printf ( "SIGCONT Received\n" ) ;
                break ;
        }
}

int main( )
{
        char buffer [ 80 ] = "\0" ;
        sigset_t block ;

        signal ( SIGTERM, sighandler ) ;
        signal ( SIGINT,  sighandler ) ;
        signal ( SIGCONT, sighandler ) ;

        sigemptyset ( &block ) ;
        sigaddset ( &block, SIGTERM ) ;
        sigaddset ( &block, SIGINT ) ;

        sigprocmask ( SIG_BLOCK, &block, NULL ) ;

        while ( strcmp ( buffer,"n" ) != 0 )
        {
            printf ( "Enter a String: " ) ;
            gets ( buffer ) ;
            puts ( buffer ) ;
        }

        sigprocmask ( SIG_UNBLOCK, &block, NULL ) ;

        while ( 1 )
            printf ( "\rProgram Running" ) ;

        return 0 ;
}
```

In this program we have registered a common handler for the **SIGINT**, **SIGTERM** and **SIGCONT** signals. Next we want to

repeatedly accept strings in a buffer and display them on the screen till the time the user does not enter an 'n' from the keyboard. Additionally, we want that this activity of receiving input should not be interrupted by the **SIGINT** or the **SIGTERM** signals. However, a **SIGCONT** should be permitted. So before we proceed with the loop we must block the **SIGINT** and **SIGTERM** signals. Once we are through with the loop we must unblock these signals. This blocking and unblocking of signals can be achieved using the **sigprocmask( )** library function.

The first parameter of the **sigprocmask( )** function specifies whether we want to block/unblock a set of signals. The next parameter is the address of a structure (**typedef**ed as **sigset_t**) that describes a set of signals that we want to block/unblock. The last parameter can be either NULL or the address of **sigset_t** type variable which would be set up with the existing set of signals before blocking/unblocking signals.

There are library functions that help us to populate the **sigset_t** structure. The **sigemptyset( )** empties a **sigset_t** variable so that it does not refer to any signals. The only parameter that this function accepts is the address of the **sigset_t** variable. We have used this function to quickly initialize the **sigset_t** variable block to a known empty state. To block the **SIGINT** and **SIGTERM** we have to add the signals to the empty set of signals. This can be achieved using the **sigaddset( )** library function. The first parameter of **sigaddset( )** is the address of the **sigset_t** variable and the second parameter is the ID of the signal that we wish to add to the existing set of signals.

After the loop we have also used an infinite **while** loop to print the 'Program running' message. This is done so that we can easily check that till the time the loop that receives input is not over the program cannot be terminated using Ctrl + C or **kill** command since the signals are blocked. Once the user enters 'n' from the keyboard the execution comes out of the **while** loop and unblocks

the signals. As a result, pending signals, if any, are immediately delivered to the program. So if we press Ctrl + C or use the **kill** command when the execution of the loop that receives input is not over these signals would be kept pending. Once we are through with the loop the signal handlers would be called.

Event Driven programming

Having understood the mechanism of signal processing let us now see how signaling is used by Linux – based libraries to create event driven GUI programs. As you know, in a GUI program events occur typically when we click on the window, type a character, close the window, repaint the window, etc. We have chosen the GTK library version 2.0 to create the GUI applications. Here, GTK stands for Gimp's Tool Kit. Refer Appendix H for installation of this toolkit. Given below is the first program that uses this toolkit to create a window on the screen.

```
/* mywindow.c */
# include <gtk/gtk.h>

int main ( int   argc, char *argv[ ] )
{
      GtkWidget *p ;

      gtk_init ( &argc, &argv ) ;
      p = gtk_window_new ( GTK_WINDOW_TOPLEVEL ) ;
      gtk_window_set_title ( p , "Sample Window" ) ;
      g_signal_connect ( p, "destroy", gtk_main_quit, NULL ) ;
      gtk_widget_set_size_request ( p, 300, 300 ) ;
      gtk_widget_show ( p ) ;
      gtk_main( ) ;

      return 0 ;
}
```